

Manual Técnico - Desarrollo, Conexión y Gestión de Contenedores en Entornos Virtualizados

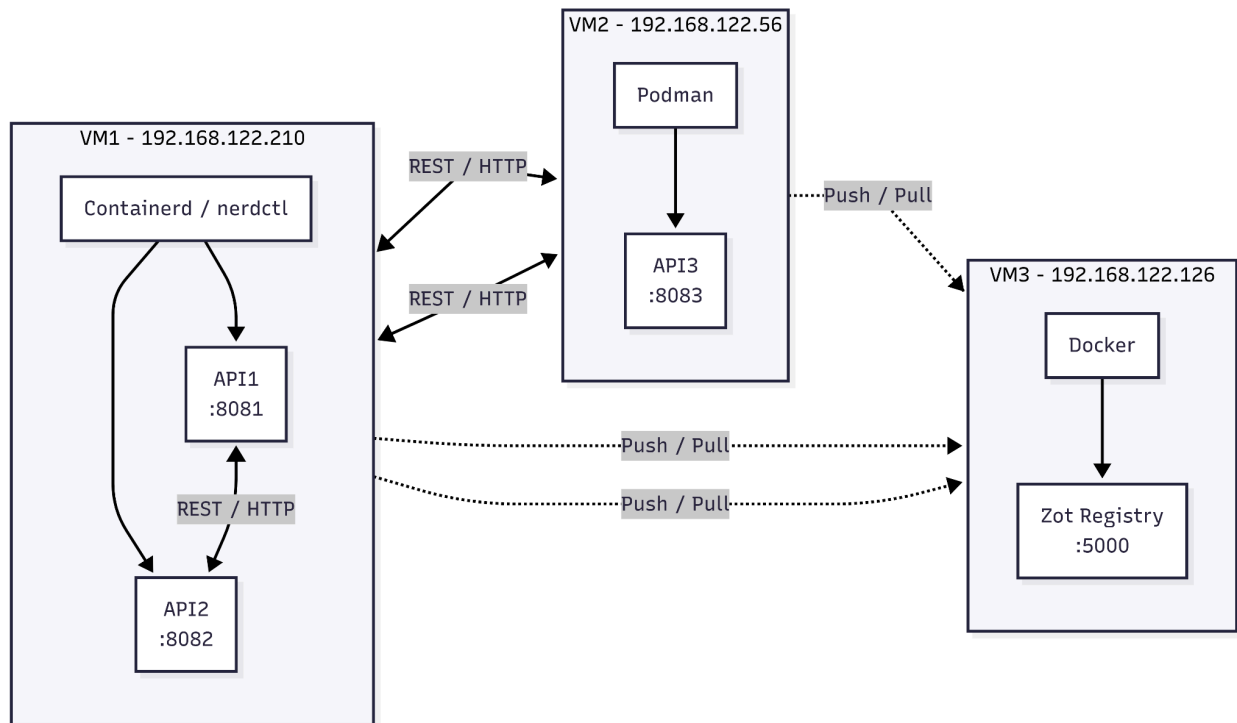
Angel Emanuel Rodriguez Corado - 202404856

Índice

Índice.....	2
Arquitectura del Proyecto.....	4
Requisitos y Configuración del Entorno.....	4
Desarrollo de las APIS en Go.....	5
VM1.....	5
API1.....	5
Dockerfile:.....	5
go.mod:.....	5
main.go:.....	6
API2.....	9
Dockerfile:.....	9
go.mod:.....	9
main.go:.....	10
VM2.....	13
API3.....	13
Dockerfile:.....	13
go.mod:.....	14
main.go:.....	14
Gestion de Imágenes y Registro Centralizado (Zot).....	17
Construcción de Imágenes.....	17
VM1 (Usando Containerd / nerdctl para API1 y API2):.....	17
VM2 (Usando Podman para API3):.....	18
Etiquetado [API#]-[CARNET]:v1.....	18
VM1:.....	18
API1:.....	18
API2:.....	18
VM2:.....	18
API3:.....	18
Subida de Imágenes a la Bodega (Push).....	18
VM1 (con nerdctl):.....	18
VM2 (con podman y omisión de verificación TLS por uso de HTTP plano):.....	19
Como arrancar el Proyecto:.....	19
Paso 1: En la VM3 (Bodega Zot).....	19
Paso 2: En la VM1 (API1 y API2).....	19
Paso 3: En la VM2 (API3).....	20
Paso 4: Comunicación Cruzada.....	21
VM1.....	21
VM2.....	21
Evidencia Visual.....	22

Verificación del Catálogo General de Repositorios.....	22
Verificación del Listado de Tags por Repositorio.....	22
API1 llamando a API2 y API3.....	23
API2 llamando a API1 y API3.....	24
API3 llamando a API1 y API2.....	24
Hacer Pull desde la VM2 (descargando imágenes creadas en la VM1).....	25
Errores Comunes y Soluciones.....	26
1. Error de Certificados / HTTP vs. HTTPS en el Registro OCI (Zot).....	26
2. Contenedor Inexistente Tras el Reinicio de la Máquina Virtual.....	26

Arquitectura del Proyecto



Requisitos y Configuración del Entorno

- Instalación y configuración de KVM en las VMs.
- Instalación de Go (Golang 1.22) en los nodos de desarrollo.
- Configuración de los runtimes de contenedores (containerd/nerdctl en VM1 y Podman en VM2).

Desarrollo de las APIS en Go

VM1

API1

Dockerfile:

```
# Etapa de compilación
FROM golang:1.22-alpine AS builder

WORKDIR /app

# Copiar dependencias y código fuente
COPY go.mod ./
COPY main.go ./

# Compilar la aplicación estáticamente
RUN CGO_ENABLED=0 GOOS=linux go build -o api1 main.go

# Etapa final ligera
FROM alpine:latest

WORKDIR /root/

# Copiar el binario compilado desde la etapa anterior
COPY --from=builder /app/api1 .

EXPOSE 8081

CMD ["/api1"]
```

go.mod:

```
module api1

go 1.22.0
```

[main.go](#):

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "time"
)

// Confi de API1
const (
    carnet = "202404856"
    vm      = "VM1"
    port    = ":8081"
)

// HealthResponse define la estructura para el endpoint de salud
type HealthResponse struct {
    Status      string `json:"status"`
    Message     string `json:"message"`
    Timestamp   string `json:"timestamp"`
    VM          string `json:"VM"`
    Carnet      string `json:"carnet"`
}

// ConnectionResponse define la estructura exacta solicitada por el
// proyecto para las llamadas cruzadas
type ConnectionResponse struct {
    APIName     string `json:"apiname"`
    Message     string `json:"message"`
    Connection  bool   `json:"connection"`
    Carnet      string `json:"carnet"`
}

// healthHandler maneja la peticion de estado de la API1
func healthHandler(w http.ResponseWriter, r *http.Request) {
    response := HealthResponse{
        Status:      "UP",
        Message:     "API1 is Ready",
        Timestamp:   time.Now().UTC().Format(time.RFC3339),
    }
```

```

        VM:      vm,
        Carnet:   carnet,
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(response)
}

// callAPI2Handler realiza la llamada hacia la API2 ubicada en la VM1 (192.168.122.210)
func callAPI2Handler(w http.ResponseWriter, r *http.Request) {
    checkAPI(w, "API2", "VM1", "http://192.168.122.210:8082/health")
}

// callAPI3Handler realiza la llamada hacia la API3 ubicada en la VM2 (192.168.122.56)
func callAPI3Handler(w http.ResponseWriter, r *http.Request) {
    checkAPI(w, "API3", "VM2", "http://192.168.122.56:8083/health")
}

// checkAPI envia una peticion HTTP GET para validar si la API de destino esta activa
func checkAPI(w http.ResponseWriter, apiName string, targetVM string, url string) {
    client := http.Client{
        Timeout: 5 * time.Second,
    }

    response, err := client.Get(url)

    if err != nil {
        sendConnectionResponse(w, apiName, targetVM, false)
        return
    }

    defer response.Body.Close()

    if response.StatusCode == http.StatusOK {
        var health HealthResponse
        err := json.NewDecoder(response.Body).Decode(&health)

        if err == nil && health.Status == "UP" {
            sendConnectionResponse(w, apiName, targetVM, true)
        }
    }
}

```

```

        return
    }
}

sendConnectionResponse(w, apiName, targetVM, false)
}

// sendConnectionResponse arma la respuesta JSON detallando la VM correspondiente segun el enunciado
func sendConnectionResponse(w http.ResponseWriter, apiName string, targetVM string, connected bool) {
    var message string

    if connected {
        message = fmt.Sprintf("The %s located on the %s is working",
apiName, targetVM)
    } else {
        message = fmt.Sprintf("ERROR: The %s located on the %s is not
working", apiName, targetVM)
    }

    response := ConnectionResponse{
        APIName:    apiName,
        Message:    message,
        Connection: connected,
        Carnet:     carnet,
    }

    w.Header().Set("Content-Type", "application/json")

    if !connected {
        w.WriteHeader(http.StatusServiceUnavailable)
    }

    json.NewEncoder(w).Encode(response)
}

// main configura los endpoints y levanto el servidor HTTP en el puerto indicado
func main() {
    http.HandleFunc("/health", healthHandler)
    http.HandleFunc("/api1/"+carnet+"/call-api2", callAPI2Handler)
    http.HandleFunc("/api1/"+carnet+"/call-api3", callAPI3Handler)
}

```

```
log.Printf("API1 corriendo en el puerto %s", port)
log.Fatal(http.ListenAndServe(port, nil))
}
```

API2

Dockerfile:

```
# Etapa de compilacion
FROM golang:1.22-alpine AS builder

WORKDIR /app

# Copiar dependencias y código fuente
COPY go.mod ./
COPY main.go ./

# Compilar la aplicacion estaticamente
RUN go build -o api2 main.go

# Etapa final ligera
FROM alpine:latest

WORKDIR /root/

# Copiar el binario compilado desde la etapa anterior
COPY --from=builder /app/api2 .

EXPOSE 8082

CMD [ "./api2" ]
```

go.mod:

```
module ap2

go 1.22.0
```

[main.go](#):

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "time"
)

// CONfi de api2 y Las rutas
const (
    carnet = "202404856"
    vm      = "VM1"
    port    = ":8082"

    api1URL = "http://192.168.122.210:8081"
    api3URL = "http://192.168.122.56:8083"
)

// Estructura de health
type HealthResponse struct {
    Status      string `json:"status"`
    Message     string `json:"message"`
    Timestamp   string `json:"timestamp"`
    VM          string `json:"VM"`
    Carnet      string `json:"carnet"`
}

// Estructura que uso cuando intento comunicarme con otra API
type ConnectionResponse struct {
    APIName     string `json:"apiname"`
    Message     string `json:"message"`
    Connection  bool   `json:"connection"`
    Carnet      string `json:"carnet"`
}

// Endpoint de salud
func healthHandler(w http.ResponseWriter, r *http.Request) {
    response := HealthResponse{
        Status:    "UP",
    }
```

```

        Message:  "API2 is Ready",
        Timestamp: time.Now().UTC().Format(time.RFC3339),
        VM:       vm,
        Carnet:   carnet,
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(response)
}

// Manejador para Llamar a La API1
func callAPI1Handler(w http.ResponseWriter, r *http.Request) {
    checkAPI(w, "API1", "VM1", api1URL+"/health")
}

// Manejador para Llamar a La API3
func callAPI3Handler(w http.ResponseWriter, r *http.Request) {
    checkAPI(w, "API3", "VM2", api3URL+"/health")
}

// Funcion generica donde valido si La API destino responde o no
func checkAPI(w http.ResponseWriter, apiName string, targetVM string, url
string) {
    client := http.Client{
        Timeout: 5 * time.Second,
    }

    response, err := client.Get(url)

    // Si fallo la peticion, asumo que no hay conexion
    if err != nil {
        sendConnectionResponse(w, apiName, targetVM, false)
        return
    }

    defer response.Body.Close()

    // Si la respuesta fue exitosa (200 OK) y el estado es UP, marco
    // conexion exitosa
    if response.StatusCode == http.StatusOK {
        var health HealthResponse

        err := json.NewDecoder(response.Body).Decode(&health)

```

```

        if err == nil && health.Status == "UP" {
            sendConnectionResponse(w, apiName, targetVM, true)
            return
        }
    }

    sendConnectionResponse(w, apiName, targetVM, false)
}

// Construyo y envio la respuesta JSON indicando el estado
func sendConnectionResponse(w http.ResponseWriter, apiName string, targetVM
string, connected bool) {
    var message string

    if connected {
        message = fmt.Sprintf(
            "The %s located on the %s is working",
            apiName,
            targetVM,
        )
    } else {
        message = fmt.Sprintf(
            "ERROR: The %s located on the %s is not working",
            apiName,
            targetVM,
        )
    }

    response := ConnectionResponse{
        APIName:    apiName,
        Message:     message,
        Connection:  connected,
        Carnet:      carnet,
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(response)
}

func main() {
    // Configuro mis rutas de endpoints
    http.HandleFunc("/health", healthHandler)
}

```

```

    http.HandleFunc(
        "/api2/"+carnet+"/call-api1",
        callAPI1Handler,
    )
    http.HandleFunc(
        "/api2/"+carnet+"/call-api3",
        callAPI3Handler,
    )
    log.Printf("API2 running on %s", port)
    // Arranco mi servidor en el puerto 8082
    log.Fatal(http.ListenAndServe(port, nil))
}

```

VM2

API3

Dockerfile:

```

# Etapa de compilacion
FROM docker.io/golang:1.22-alpine AS builder

WORKDIR /app

# Copiar dependencias y código fuente
COPY go.mod ./
COPY main.go ./

# Compilar la aplicacion estaticamente
RUN go build -o api3 main.go

# Etapa final ligera
FROM docker.io/alpine:latest

WORKDIR /root/

COPY --from=builder /app/api3 .

EXPOSE 8083

CMD [ "./api3" ]

```

go.mod:

```
module api3

go 1.22.0
```

[main.go](#):

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "time"
)

// Confi de api3 y Las rutas
const (
    carnet = "202404856"
    vm      = "VM2"
    port    = ":8083"

    api1URL = "http://192.168.122.210:8081"
    api2URL = "http://192.168.122.210:8082"
)

// Estructura que uso para responder
type HealthResponse struct {
    Status      string `json:"status"`
    Message     string `json:"message"`
    Timestamp   string `json:"timestamp"`
    VM          string `json:"VM"`
    Carnet      string `json:"carnet"`
}

// Estructura que uso cuando intento comunicarme con otra API
type ConnectionResponse struct {
    APIName     string `json:"apiname"`
    Message     string `json:"message"`
    Connection  bool   `json:"connection"`
}
```

```

    Carnet      string `json:"carnet"`
}

// Endpoint de salud
func healthHandler(w http.ResponseWriter, r *http.Request) {
    response := HealthResponse{
        Status:      "UP",
        Message:     "API3 is Ready",
        Timestamp:   time.Now().UTC().Format(time.RFC3339),
        VM:          vm,
        Carnet:      carnet,
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(response)
}

// Manejador para Llamar a La API1
func callAPI1Handler(w http.ResponseWriter, r *http.Request) {
    checkAPI(w, "API1", "VM1", api1URL+"/health")
}

// Manejador para Llamar a La API2
func callAPI2Handler(w http.ResponseWriter, r *http.Request) {
    checkAPI(w, "API2", "VM1", api2URL+"/health")
}

// Funcion generica donde valido si La API destino responde o no
func checkAPI(w http.ResponseWriter, apiName string, targetVM string, url string) {
    client := http.Client{
        Timeout: 5 * time.Second,
    }

    response, err := client.Get(url)

    // Si fallo la peticion, asumo que no hay conexion
    if err != nil {
        sendConnectionResponse(w, apiName, targetVM, false)
        return
    }

    defer response.Body.Close()

```

```

    // Si la respuesta fue exitosa (200 OK) y el estado es UP, marco
    // conexion exitosa
    if response.StatusCode == http.StatusOK {
        var health HealthResponse

        err := json.NewDecoder(response.Body).Decode(&health)

        if err == nil && health.Status == "UP" {
            sendConnectionResponse(w, apiName, targetVM, true)
            return
        }
    }

    sendConnectionResponse(w, apiName, targetVM, false)
}

// Construyo y envio la respuesta JSON indicando el estado de la
// comunicacion con la otra API
func sendConnectionResponse(w http.ResponseWriter, apiName string, targetVM
string, connected bool) {
    var message string

    if connected {
        message = fmt.Sprintf(
            "The %s located on the %s is working",
            apiName,
            targetVM,
        )
    } else {
        message = fmt.Sprintf(
            "ERROR: The %s located on the %s is not working",
            apiName,
            targetVM,
        )
    }

    response := ConnectionResponse{
        APIName:    apiName,
        Message:    message,
        Connection: connected,
        Carnet:     carnet,
    }
}

```

```

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(response)
}

func main() {
    // Configuro mis rutas de endpoints
    http.HandleFunc("/health", healthHandler)

    http.HandleFunc(
        "/api3/"+carnet+"/call-api1",
        callAPI1Handler,
    )

    http.HandleFunc(
        "/api3/"+carnet+"/call-api2",
        callAPI2Handler,
    )

    log.Printf("API3 running on %s", port)

    // Arranco mi servidor en el puerto 8083
    log.Fatal(http.ListenAndServe(port, nil))
}

```

Gestion de Imagenes y Registro Centralizado (Zot)

Construcción de Imágenes

Dependiendo del runtime de contenedores utilizado en cada nodo, se construyeron las imágenes locales a partir de los Dockerfile

VM1 (Usando Containerd / nerdctl para API1 y API2):

```

# Para API1
sudo nerdctl build -t api1:v1 .

# Para API2
sudo nerdctl build -t api2:v1 .

```

VM2 (Usando Podman para API3):

```
podman build -t api3:v1 .
```

Etiquetado [API#]-[CARNET]:v1

se etiquetaron las imágenes apuntando directamente hacia la dirección IP y puerto del registro Zot en la VM3

VM1:

API1:

```
sudo nerdctl tag api1:v1 192.168.122.126:5000/api1-202404856:v1
```

API2:

```
sudo nerdctl tag api2:v1 192.168.122.126:5000/api2-202404856:v1
```

VM2:

API3:

```
podman tag api3:v1 192.168.122.126:5000/api3-202404856:v1
```

Subida de Imágenes a la Bodega (Push)

Una vez etiquetadas, se procedió a subir las imágenes hacia el registro centralizado en la VM3:

VM1 (con nerdctl):

```
sudo nerdctl push 192.168.122.126:5000/api1-202404856:v1  
sudo nerdctl push 192.168.122.126:5000/api2-202404856:v1
```

VM2 (con podman y omisión de verificación TLS por uso de HTTP plano):

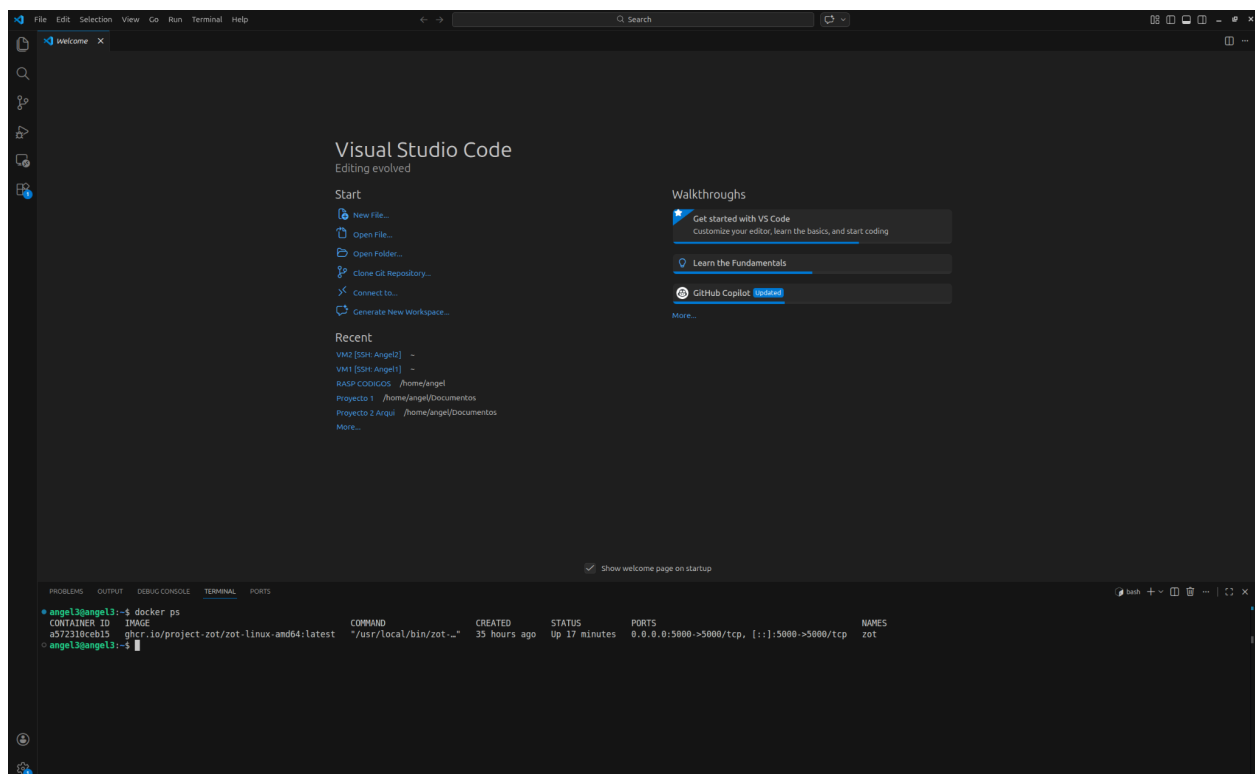
```
podman push --tls-verify=false 192.168.122.126:5000/api3-202404856:v1
```

Como arrancar el Proyecto:

Paso 1: En la VM3 (Bodega Zot)

Asegurarnos de que el registro centralizado esté corriendo para que los nodos puedan comunicarse con él si lo necesitan:

```
sudo docker start zot
```



Paso 2: En la VM1 (API1 y API2)

Conéctarnos a la VM1 (192.168.122.210) y arranca los contenedores gestionados con containerd/nerdctl:

```
# Arrancar Los contenedores existentes
sudo nerdctl start api1
sudo nerdctl start api2

# (Recomendación) Verificar que responden Localmente
curl http://localhost:8081/health
curl http://localhost:8082/health
```

The screenshot shows a VS Code editor with a Go file named `main.go`. The code defines a REST API with endpoints for `/health`, `/api1/carnet`, and `/api2/carnet`. It uses the `http.HandlerFunc` and `http.ListenAndServe` functions. The `main` function sets up the routes and starts the server on port 8082.

The terminal output shows the following commands and results:

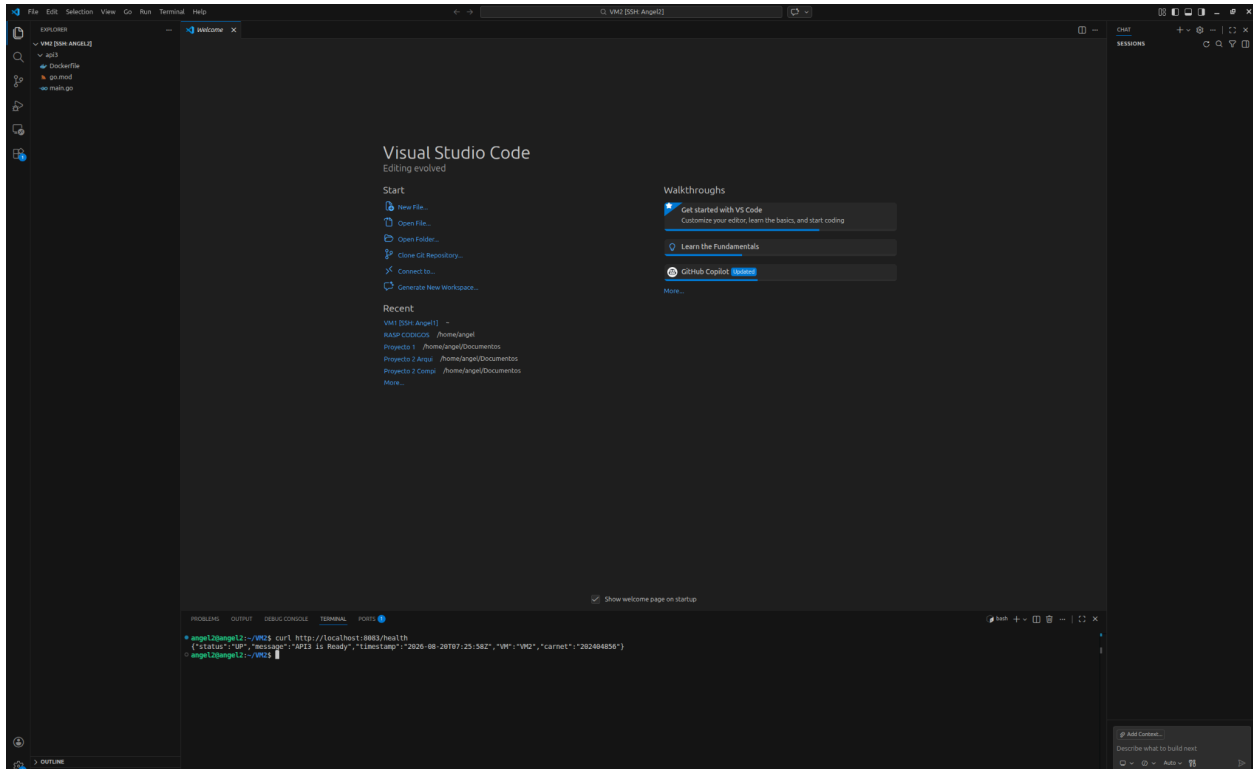
```
* angelibangel@192:~$ curl http://localhost:8081/health
{"status":"up","message":"API1 is Ready","timestamp":"2026-08-20T07:26:49Z","vm":"VM2","carnet":"262404836"}
* angelibangel@192:~$ curl http://localhost:8082/health
{"status":"up","message":"API2 is Ready","timestamp":"2026-08-20T07:27:01Z","vm":"VM2","carnet":"262404836"}
angelibangel@192:~$
```

Paso 3: En la VM2 (API3)

Conéctate a la VM2 (192.168.122.56) y arranca el contenedor gestionado con podman:

```
# Arrancar el contenedor existente
sudo podman start api3

# (Recomendación) Verificar que responde Localmente
curl http://localhost:8083/health
```



Paso 4: Comunicación Cruzada

Para comprobar que todo el sistema distribuido está al operativo y funcionando, ejecutamos estas pruebas desde cualquier terminal de las VMs:

VM1

```
curl http://192.168.122.210:8081/api1/202404856/call-api2  
curl http://192.168.122.210:8081/api1/202404856/call-api3
```

VM2

```
curl http://192.168.122.56:8083/api3/202404856/call-api1  
curl http://192.168.122.56:8083/api3/202404856/call-api2
```

Todos tienen que devolver **"connection":true**

Evidencia Visual

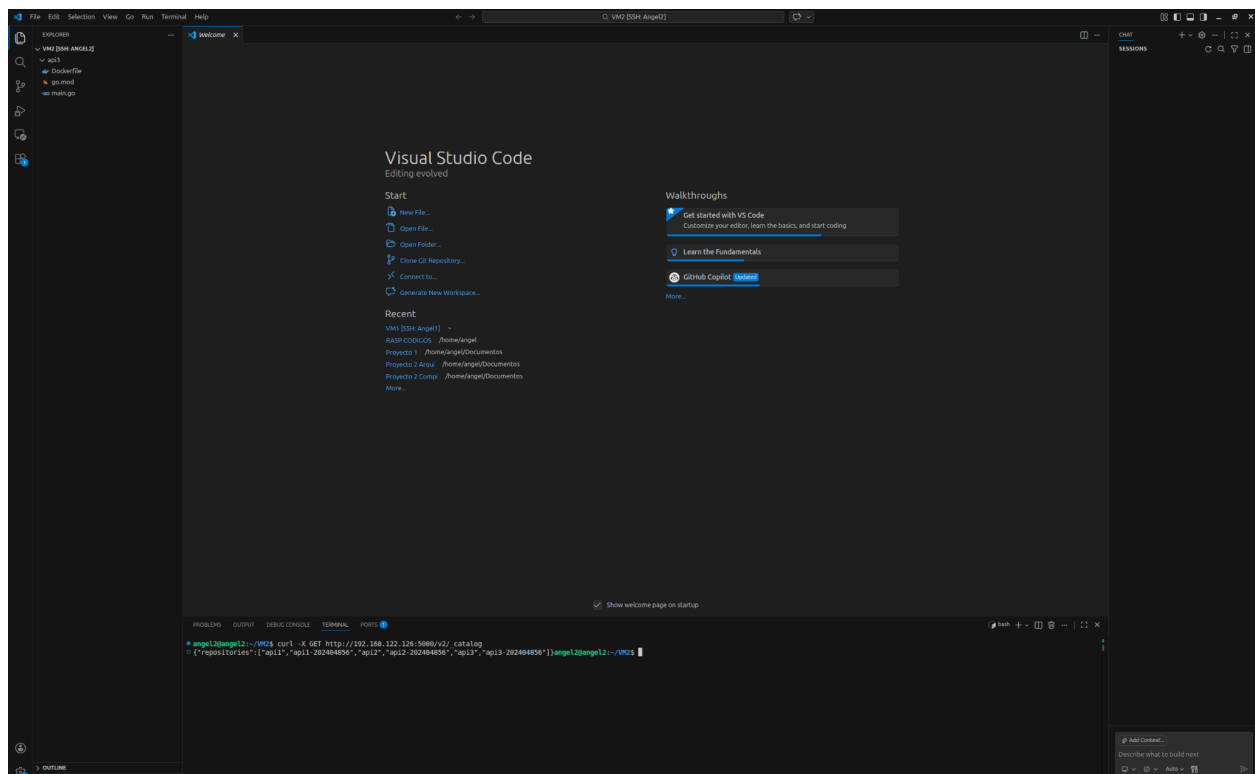
Verificación del Catálogo General de Repositorios

Comando Ejecutado:

```
curl -X GET http://192.168.122.126:5000/v2/_catalog
```

Salida obtenida:

```
{"repositories":["api1","api1-202404856","api2","api2-202404856","api3","api3-202404856"]}
```



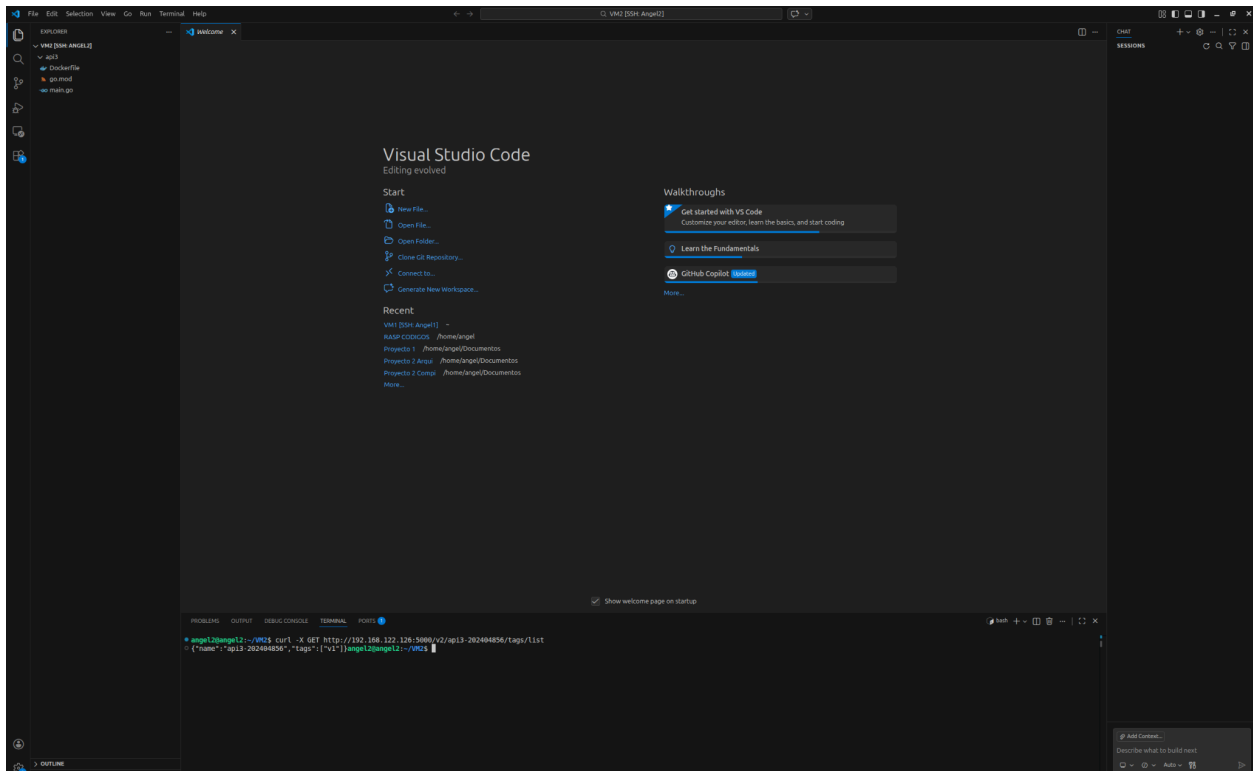
Verificación del Listado de Tags por Repositorio

Comando Ejecutado:

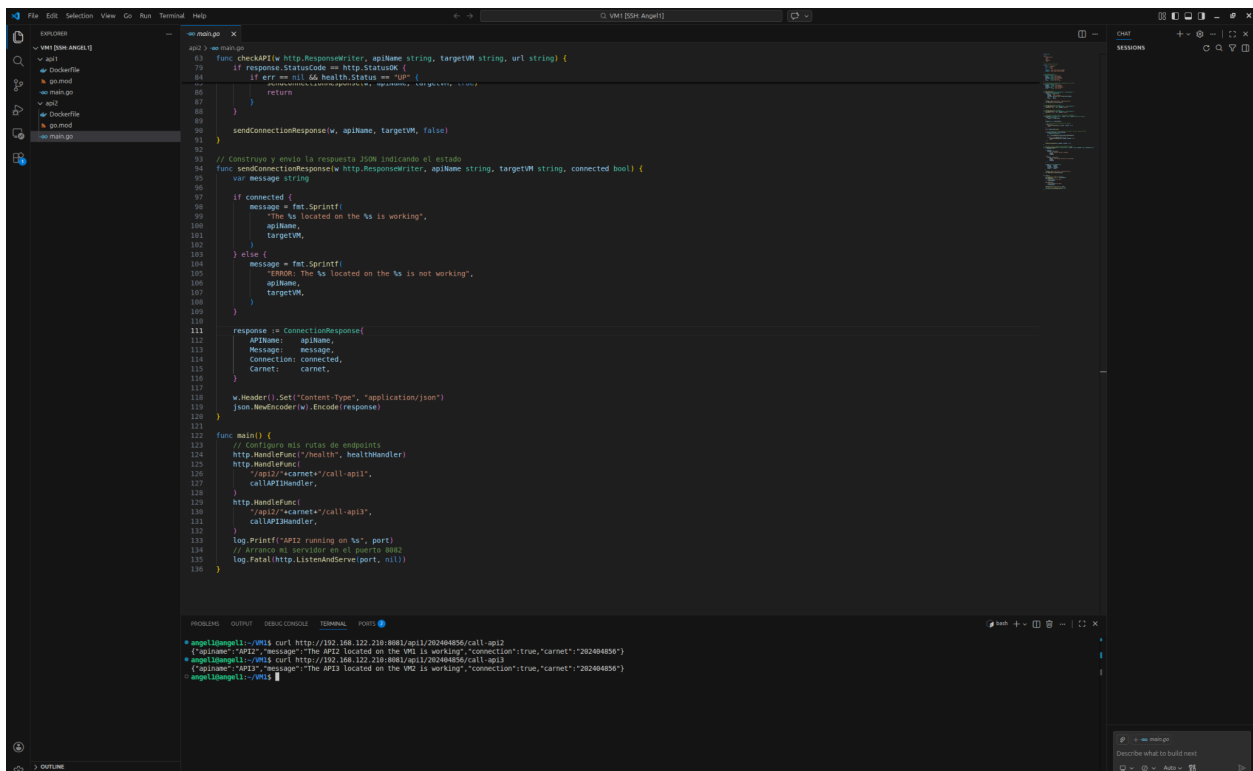
```
curl -X GET http://192.168.122.126:5000/v2/api3-202404856/tags/list
```

Salida obtenida:

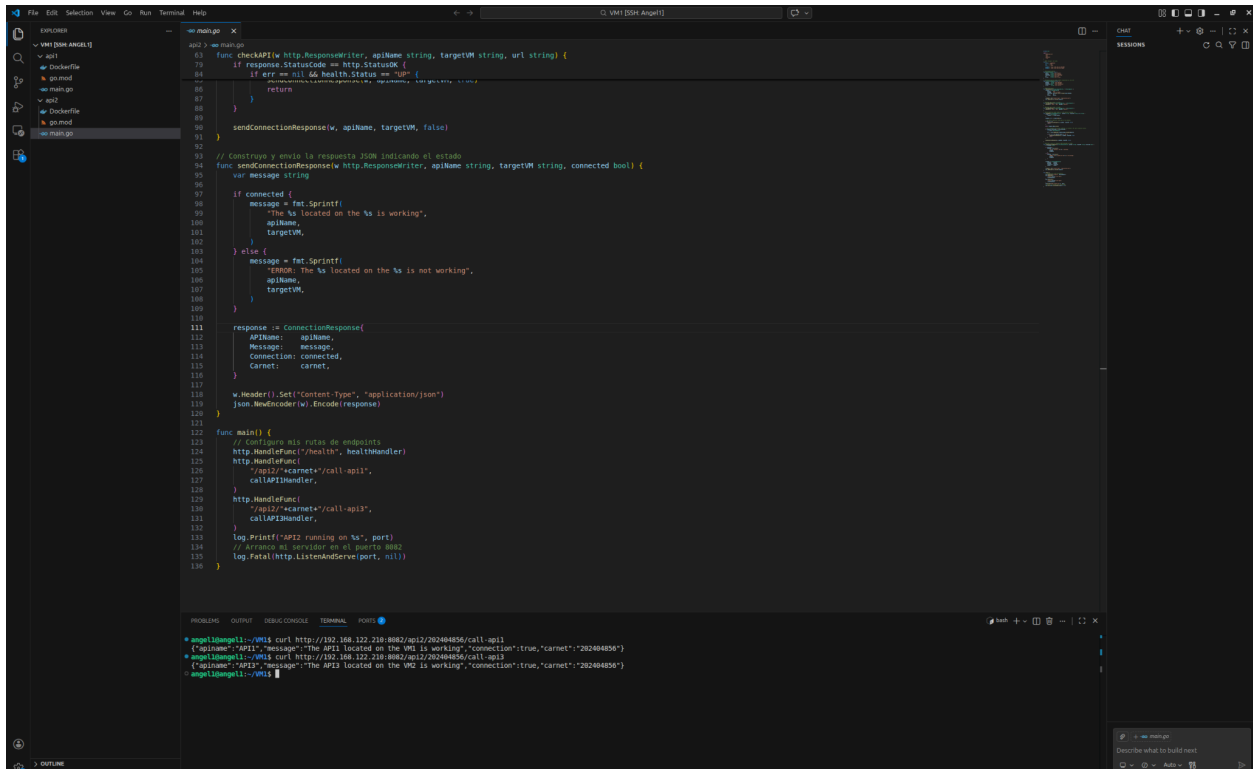
```
{"name":"api3-202404856","tags":["v1"]}
```



API1 llamando a API2 y API3



API2 llamando a API1 y API3

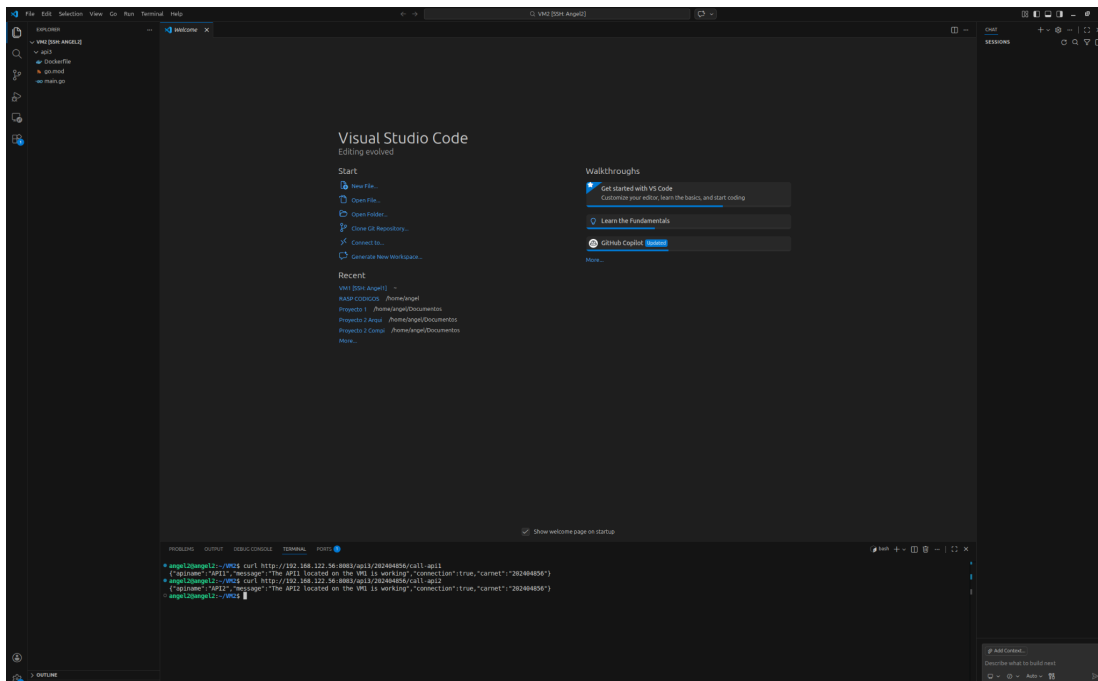


```
13 func checkAPI1(w http.ResponseWriter, apiName string, url string) {
14     if response.StatusCode == http.StatusOK {
15         if err := nil; healthStatus == "up" {
16             return
17         }
18     }
19     sendConnectionResponse(w, apiName, targetURL, false)
20 }
21
22 // Construye y envia la respuesta JSON indicando el estado
23 func sendConnectionResponse(w http.ResponseWriter, apiName string, targetURL string, connected bool) {
24     var message string
25
26     if connected {
27         message = fmt.Sprintf(
28             "The %s located on the %s is working",
29             apiName,
30             targetURL,
31         )
32     } else {
33         message = fmt.Sprintf(
34             "ERROR: The %s located on the %s is not working",
35             apiName,
36             targetURL,
37         )
38     }
39
40     response := ConnectionResponse{
41         APIName: apiName,
42         Message: message,
43         Connection: connected,
44         Carnet: carnet,
45     }
46
47     w.Header().Set("Content-Type", "application/json")
48     json.NewEncoder(w).Encode(response)
49 }
50
51 func main() {
52     // Configuramos los puertos de endpoints
53     http.HandleFunc("/health", healthHandler)
54     http.HandleFunc(
55         "/api2/"+carnet+"/call-api1",
56         callAPI1Handler,
57     )
58     http.HandleFunc(
59         "/api2/"+carnet+"/call-api3",
60         callAPI3Handler,
61     )
62     log.Printf("API2 running on %s", port)
63     // Arranca el servidor en el puerto 8082
64     log.Fatal(http.ListenAndServe(port, nil))
65 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
angel@angel1:~/API1$ curl http://192.168.122.218:8082/api2/202404056/call-api1
{"APIName": "API1", "Message": "The API2 located on the VM1 is working", "Connection": true, "Carnet": "202404056"}
angel@angel1:~/API1$ curl http://192.168.122.218:8082/api2/202404056/call-api3
{"APIName": "API3", "Message": "The API2 located on the VM2 is working", "Connection": true, "Carnet": "202404056"}
angel@angel1:~/API1$
```

API3 llamando a API1 y API2



Visual Studio Code
Editing evolved

Start

- New File...
- Open File...
- Open Folder...
- Open Recent...
- Connect To...
- Generate New Workspace...

Recent

- VM1 [SSH Angel1]
- API1 [SSH Angel1]
- API2 [SSH Angel1]
- API3 [SSH Angel1]

Walkthroughs

- Get started with VS Code
- Learn the Fundamentals
- GitHub Copilot

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
angel@angel1:~/API1$ curl http://192.168.122.218:8082/api2/202404056/call-api1
{"APIName": "API1", "Message": "The API2 located on the VM1 is working", "Connection": true, "Carnet": "202404056"}
angel@angel1:~/API1$ curl http://192.168.122.218:8082/api2/202404056/call-api2
{"APIName": "API2", "Message": "The API2 located on the VM1 is working", "Connection": true, "Carnet": "202404056"}
angel@angel1:~/API1$
```

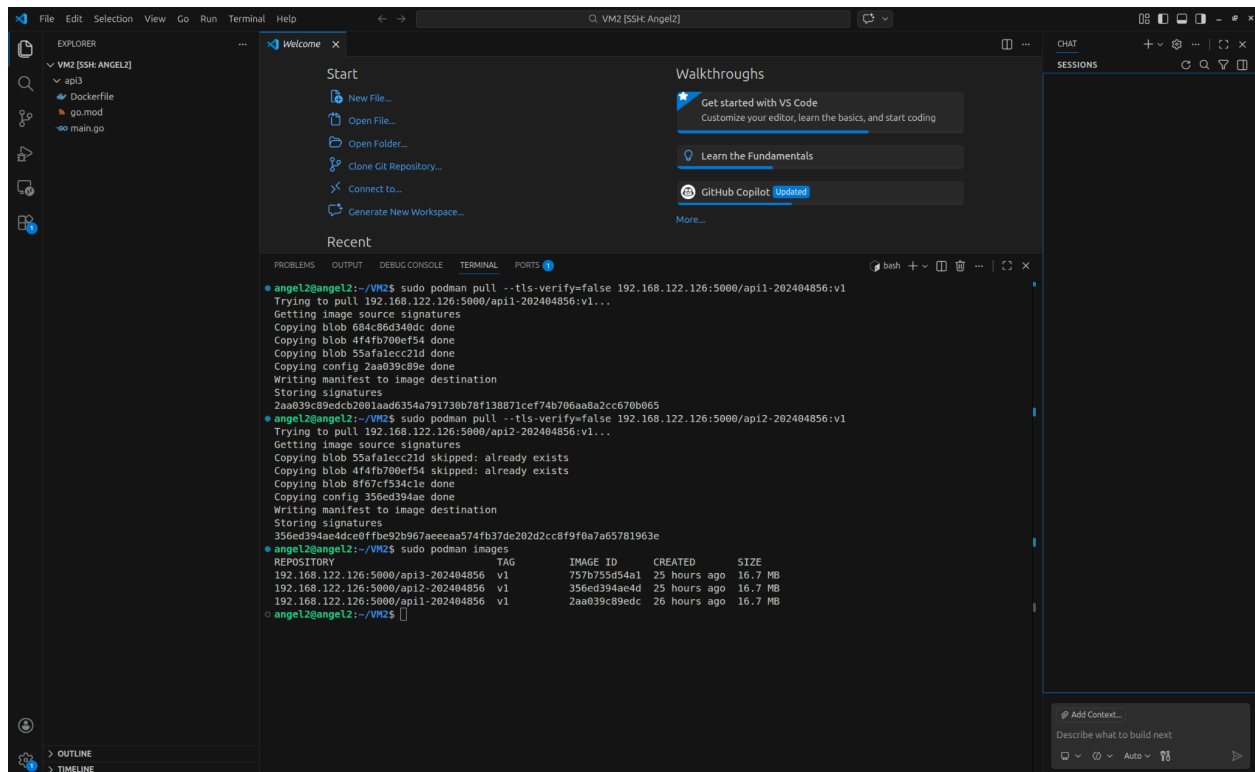
Hacer Pull desde la VM2 (descargando imágenes creadas en la VM1)

Descargar la API1 con tu carnet

```
sudo podman pull --tls-verify=false 192.168.122.126:5000/api1-202404856:v1
```

Descargar la API2 con tu carnet

```
sudo podman pull --tls-verify=false 192.168.122.126:5000/api2-202404856:v1
```



The screenshot shows a VS Code editor window with a terminal open. The terminal displays the execution of two podman pull commands. The first command pulls the image 192.168.122.126:5000/api1-202404856:v1, and the second command pulls 192.168.122.126:5000/api2-202404856:v1. The output shows the progress of downloading the image layers and writing the manifest. Below the commands, the podman images command is run, displaying a table of the pulled images.

```
angel2@angel2:~/VM2$ sudo podman pull --tls-verify=false 192.168.122.126:5000/api1-202404856:v1
Trying to pull 192.168.122.126:5000/api1-202404856:v1...
Getting image source signatures
Copying blob 684c86d340dc done
Copying blob 4f4fb700cf54 done
Copying blob 55afalecc21d done
Copying config 2aa039c89e done
Writing manifest to image destination
Storing signatures
2aa039c89edcb2001aad6354a791730b78f138871cef74b706aa8a2cc670b065

angel2@angel2:~/VM2$ sudo podman pull --tls-verify=false 192.168.122.126:5000/api2-202404856:v1
Trying to pull 192.168.122.126:5000/api2-202404856:v1...
Getting image source signatures
Copying blob 55afalecc21d skipped: already exists
Copying blob 4f4fb700cf54 skipped: already exists
Copying blob 8f67cf534c1e done
Copying config 356ed394ae done
Writing manifest to image destination
Storing signatures
356ed394ae4dce0fb92b967aeeaa574fb37de202d2cc8f9f0a7a65781963e

angel2@angel2:~/VM2$ sudo podman images
REPOSITORY          TAG          IMAGE ID      CREATED      SIZE
192.168.122.126:5000/api3-202404856  v1          757b755d54a1  25 hours ago  16.7 MB
192.168.122.126:5000/api2-202404856  v1          356ed394ae4d  25 hours ago  16.7 MB
192.168.122.126:5000/api1-202404856  v1          2aa039c89edc  26 hours ago  16.7 MB
```

Errores Comunes y Soluciones

Durante el despliegue de la infraestructura distribuida y la gestión de contenedores, se identificaron y solucionaron los siguientes incidentes técnicos:

1. Error de Certificados / HTTP vs. HTTPS en el Registro OCI (Zot)

- **Descripción del problema:** Al intentar realizar un comando `push` o `pull` hacia el registro centralizado (Zot) alojado en la VM3 (`192.168.122.126:5000`), el runtime de contenedores (Podman/Containerd) arrojaba un error de rechazo indicando que el servidor respondía con HTTP plano pero el cliente esperaba HTTPS.
 - *Mensaje de error típico:* `Error: initializing source docker://...: http: server gave HTTP response to HTTPS client`
- **Causa:** Por defecto, los runtimes modernos de contenedores imponen el uso de conexiones cifradas (HTTPS) por motivos de seguridad, mientras que la bodega Zot de pruebas operaba en un entorno local mediante HTTP plano.
- **Solución aplicada:** Se indicó explícitamente al cliente que omitiera la verificación de seguridad TLS mediante el parámetro de exclusión correspondiente:
 - *En Podman:* Añadir el flag `--tls-verify=false` (ej. `podman push/pull --tls-verify=false ...`).

2. Contenedor Inexistente Tras el Reinicio de la Máquina Virtual

- **Descripción del problema:** Al apagar las máquinas virtuales (KVM) y volver a encenderlas, al intentar iniciar los contenedores existentes mediante `podman start api3`, el sistema informaba que el contenedor no se encontraba registrado.
 - *Mensaje de error típico:* `Error: no container with name or ID "api3" found: no such container`
- **Causa:** Tras un reinicio del sistema operativo anfitrión, los contenedores detenidos que no fueron configurados con una política de reinicio persistente (`restart: always`) pueden ser limpiados o no se preservan en ejecución.
- **Solución aplicada:** En lugar de intentar arrancarlos, se recreó y ejecutó el contenedor directamente utilizando la imagen oficial previamente respaldada en el registro Zot:

```
sudo podman run -d --name api3 --net=host --tls-verify=false
192.168.122.126:5000/api3-202404856:v1
```